JShotz User Guide

Version 3.11.2

JShotz is a browser extension for Chrome, Edge, and Firefox that records a browsing flow as a sequence of timestamped, watermarked screenshots and exports them as a PDF, with an optional table of API calls captured under each step. It's built for documenting test flows, support tickets, and step-by-step evidence of what happened in a browser session.

---

1. Installing the extension

Chrome / Edge

- Unzip `JShotz-3.11.2-chrome-edge.zip`.
- Go to `chrome://extensions` (or `edge://extensions`).
- Turn on **Developer mode** (top-right toggle).
- Click **Load unpacked** and select the unzipped folder that contains `manifest.json`.
- If updating, remove or disable the old version first so only one JShotz copy is loaded.

Firefox

- Unzip `JShotz-3.11.2-firefox.zip`.
- Go to `about:debugging#/runtime/this-firefox`.
- Click **Load Temporary Add-on** and select the `manifest.json` inside the unzipped folder.
- Firefox 128+ is required.

Once loaded, pin the JShotz icon to your toolbar for easy access.

Updating JShotz

After reloading or updating the extension, refresh every tab that was already open before you record with it. Closing and reopening the target tab is the most reliable option. Chrome keeps the old page script alive until the tab reloads and may show **Extension context invalidated** in the extension's error list. Delete that old error card after refreshing the tab; it does not indicate a problem with a newly opened or refreshed recording page.

---

2. The popup

Click the JShotz icon to open the popup. It has three parts:

- **Status line** — shows *Idle*, *Recording · N screenshot(s)*, or an error message.
- **Settings** — capture source and behavior options (locked once recording starts, except the

capture source, which can be changed mid-recording).

- **Actions** — Start/Stop, Capture now, Capture in 5s, Export PDF so far.
- **Captures list** — a live list of what's been captured so far in the current session.

---

3. Capture source (modes)

Mode	What it captures	Notes
Tab viewport only	The visible area of the recorded tab	Default mode, works everywhere
API + Screenshot	Same as Tab, plus a table of intercepted `fetch`/XHR calls under each screenshot	Shows request URL, origin/referer, payload, and response; failed calls are highlighted
Screen / window (DevTools + taskbar)	A shared screen, window, or tab surface via the browser's own share picker	The only mode that can show DevTools, the taskbar, or other applications

You can switch modes without stopping the recording. Pick a different option from the dropdown at any time. Switching *to* Screen/window mode opens a small picker window — click **Share** in it and choose what to share. The tab being recorded is automatically brought to the front first, since the picker needs a real click.

If Screen/window mode's share source ends (you click "Stop sharing", or close the shared window), the recording automatically falls back to Tab-viewport captures rather than failing.

---

4. Settings

- **Capture on every button click** — automatically screenshot after clicks on buttons, links, checkboxes, and similar controls.
- **Capture every 60% of a screen scrolled, and at the end** — automatically screenshot as you scroll, roughly every 60% of a screenful (so consecutive shots overlap), plus one at the very bottom of the page.
- **Stamp clock + time zone on each shot** — adds a small timestamp banner to each screenshot.
- **Whole-page shot for Ctrl+Alt+Q and "Capture now" only** — see Section 6.
- **Save individual PNG files** — save each screenshot as its own PNG in the session folder.
- **Save PDF on stop** — build a PDF from the session when you stop and choose to keep it.

---

5. Starting, capturing, and stopping

- Open the tab you want to record, then click **Start recording** in the popup.

- Use the page normally. Screenshots are taken automatically per your settings, and you can

also trigger one manually at any time (see below).

- When a page you're recording opens a **new tab** (e.g. a sign-in redirect), JShotz follows

it automatically — that tab is brought to the front and becomes part of the recording. Switching back to the original tab (or to any other tab that flow has opened) resumes capturing from wherever you actually are.

- When you're done, click **Stop recording**. You'll be asked whether to keep the files:

- **Yes, keep** — prompts for a PDF file name, then saves everything (PNGs, manifest, PDF) to your Downloads folder and opens that folder.
- **No, delete all** — asks you to confirm, then removes everything from that session.

Manual capture options

Action	How	Notes
Capture now	Click **Capture now** in the popup	Immediate, uses whichever mode is active
Manual hotkey	Press **Ctrl+Alt+Q** while the page has focus	Works anywhere the page can receive keystrokes
DevTools panel capture	Press **Alt+Shift+S** while a DevTools panel is open	Captures exactly what's on screen, including the DevTools panel
Capture in 5s	Click **Capture in 5s**, or press **Alt+Shift+D**	Waits 5 seconds (with an on-page countdown badge) before capturing — use this when you need time to click into DevTools first, since Chrome blocks other shortcuts while DevTools has focus

Export PDF so far

Click **Export PDF so far** at any point during a recording to write a checkpoint PDF from everything captured up to that moment, without stopping. The file is named `..._checkpoint.pdf` and does **not** open your Downloads folder automatically — only the final "Stop recording" export does that. The recording keeps going afterward, and the final PDF (on Stop) still includes everything, checkpoints included.

Create PDF from saved screenshots

If you have a folder of previously saved PNGs (from a session, or from anywhere), click **Create PDF from saved screenshots** in the popup to pick that folder and build a fresh PDF from its contents, independent of any active recording.

---

6. Full-page (whole-page) capture

When **"Whole-page shot"** is enabled, the following triggers capture the *entire* scrollable page instead of just the visible area — **without visibly scrolling your screen**:

- **Ctrl+Alt+Q**
- **Capture now**
- **Capture in 5s**

Everything else (clicks, scrolling, field edits, navigation) stays as ordinary viewport shots, so your view is never disturbed by the automatic captures.

How it works, and what to expect:

- For a page whose content naturally extends below the viewport, the browser briefly resizes its internal layout to render everything, takes one shot, then restores it — you may see a very brief flicker.
- For an "app-shell" style page (a fixed header/sidebar with an inner scrolling panel), JShotz scrolls just that inner panel and stitches the results — this doesn't touch your window size at all.
- If DevTools is already open on the tab, whole-page capture is skipped for that shot (DevTools and the extension can't share the same debugging connection) and a normal single-frame screenshot is taken instead.
- Chrome shows a **"started debugging this browser"** banner for the brief moment a whole-page capture is in progress. This is a hard Chrome platform notice with no way to hide it — it disappears again immediately after each shot.

---

7. Keyboard shortcuts

Shortcut	Action
`Ctrl+Alt+Q`	Manual capture (page must have focus)
`Alt+Shift+S`	Capture the current DevTools panel
`Alt+Shift+D`	Capture in 5 seconds

All three can be reassigned at `chrome://extensions/shortcuts` (or the Firefox equivalent).

---

8. Where your files go

Each recording creates a folder in your browser's default download location:

```
Downloads/
  flow-captures/
    session_<timestamp>/
      001_<timestamp>_<label>.png
      002_<timestamp>_<label>.png
      ...
      flow-manifest.json        (list of every capture: time, reason, URL, mode)
      session_<timestamp>.pdf   (if "Save PDF" was on)
      session_<timestamp>_checkpoint.pdf   (if you used "Export PDF so far")
      debug-log.txt             (diagnostic log, see below)
```

---

9. The debug log

Every session writes a `debug-log.txt` alongside the screenshots. It's not shown in the UI, but it's useful if something looks wrong and needs troubleshooting: it lists every capture attempt with its trigger, capture mode, success/failure, and timing, plus mode switches and errors. If you ever need help diagnosing an issue, this file (or the relevant lines from it) is the most useful thing to share.

The log survives even if you choose "delete all" for a session's screenshots, so it's still available afterward if something needs investigating.

---

10. Known limitations

- The "started debugging this browser" banner during whole-page captures cannot be hidden — this is a Chrome security notice, not a bug.
- Whole-page capture is skipped while DevTools is open on the recorded tab.
- Multi-tab following works for the current session only; it doesn't persist across a browser restart or an unlikely mid-session extension reload.
- A horizontally-scrolling element on a page (e.g. a carousel) is captured exactly as it appears at the time — whole-page capture only extends vertically.
- Reloading or updating JShotz requires refreshing any already-open recording tabs before use.

---

11. Getting help

If something isn't working as expected:

- Note the approximate time and what you were doing (which button, which page).
- Open `debug-log.txt` from that session's folder and find the matching lines.
- Share the page URL (or a general description if it's private), the log excerpt, and — if

relevant — the screenshot in question.